# TECHNICAL DOCUMENTATION

## RAG Customer Support Assistant

LangGraph + ChromaDB + Ollama + HITL

Complete technical explanation of system architecture, design decisions, workflow execution, routing logic, testing strategy, trade-offs, and future roadmap.

Prepared By: **Bhargav S**

April 22, 2026

# Contents

**1 Introduction** **3**
1.1 What is RAG . . . . . 3
1.2 Why it is Needed . . . . . 3
1.3 Use Case Overview . . . . . 3

**2 System Architecture Explanation** **4**
2.1 Detailed Explanation of HLD . . . . . 4
2.2 Component Interactions . . . . . 4

**3 Design Decisions** **4**
3.1 Chunk Size Choice . . . . . 4
3.2 Embeddings Strategy . . . . . 4
3.3 Retrieval Approach . . . . . 5
3.4 Prompt Design Logic . . . . . 5

**4 Workflow Explanation** **6**
4.1 LangGraph Usage . . . . . 6
4.2 Node Responsibilities . . . . . 6
4.3 State Transitions . . . . . 6

**5 Conditional Logic** **6**
5.1 Intent Detection . . . . . 6
5.2 Routing Decisions . . . . . 6

**6 HITL Implementation** **8**
6.1 Role of Human Intervention . . . . . 8
6.2 Benefits . . . . . 8
6.3 Limitations . . . . . 8

**7 Challenges & Trade-offs** **8**
7.1 Retrieval Accuracy vs Speed . . . . . 8
7.2 Chunk Size vs Context Quality . . . . . 8
7.3 Cost vs Performance . . . . . 8

**8 Testing Strategy** **9**
8.1 Testing Approach . . . . . 9
8.2 Sample Queries . . . . . 9
8.3 Validation Metrics . . . . . 9

**9 Future Enhancements** **9**

**10 Summary** **10**

# 1 Introduction

## 1.1 What is RAG

Retrieval-Augmented Generation (RAG) is an AI architecture that combines:

- Information Retrieval
- Large Language Models
- Context-aware Answer Generation

Instead of answering only from model memory, the system first retrieves relevant knowledge and then generates responses.

## 1.2 Why it is Needed

- Reduces hallucinations
- Improves factual accuracy
- Uses company-specific knowledge
- Provides explainable answers with sources
- Better than generic chatbots

## 1.3 Use Case Overview

This project is designed for customer support automation where users ask questions related to:

- Orders
- Shipping
- Refunds
- Returns
- Policies
- Account issues

# 2 System Architecture Explanation

## 2.1 Detailed Explanation of HLD

The system follows a retrieval-first pipeline:

User Query → Embed Query → Search ChromaDB → Retrieve Chunks → Generate Response using Mistral → Confidence Check → Final Answer / HITL

## 2.2 Component Interactions

- PDF documents are converted into chunks
- Chunks are embedded and stored in ChromaDB
- Query is converted into embedding
- Retriever finds relevant chunks
- LLM generates contextual response
- Router checks confidence score
- Low confidence requests human intervention

# 3 Design Decisions

## 3.1 Chunk Size Choice

Chunk Size = 500
Overlap = 100

Chosen to balance context preservation and retrieval speed.

## 3.2 Embeddings Strategy

Uses:

sentence-transformers/all-MiniLM-L6-v2

Reason:

- Fast generation
- Good semantic understanding

- Lightweight
- Free and open-source

## 3.3 Retrieval Approach

Similarity search is used to retrieve top-k chunks.

Top-k = 3

This minimizes noise while keeping relevant context.

## 3.4 Prompt Design Logic

Prompt contains:

- User query
- Retrieved context
- Instruction to answer only from context
- Avoid unsupported claims

# 4 Workflow Explanation

## 4.1 LangGraph Usage

LangGraph is used to orchestrate node-based execution flow.

## 4.2 Node Responsibilities

- Input Node: Accept query
- Retrieval Node: Search vector database
- LLM Node: Generate answer
- Router Node: Confidence decision
- HITL Node: Human escalation

## 4.3 State Transitions

Query State → Retrieved State → Response State → Routed State

Each node updates runtime state and passes it forward.

# 5 Conditional Logic

## 5.1 Intent Detection

The system interprets whether the query is:

- Simple factual query
- Missing-context query
- Complex support issue
- Sensitive issue needing review

## 5.2 Routing Decisions

**Direct Answer if:**

- Relevant chunks found
- Confidence score high
- Clear response generated

**Escalate if:**

- Low confidence
- No sources found
- Ambiguous request
- LLM uncertainty

# 6 HITL Implementation

## 6.1 Role of Human Intervention

Human reviewers validate or replace low-confidence AI responses.

## 6.2 Benefits

- Higher trust
- Prevents wrong answers
- Useful for complex queries
- Better customer experience

## 6.3 Limitations

- Requires human availability
- Slower than auto-response
- Increased operational effort

# 7 Challenges & Trade-offs

## 7.1 Retrieval Accuracy vs Speed

Higher top-k may improve recall but increases latency.

## 7.2 Chunk Size vs Context Quality

Large chunks preserve meaning but may reduce precision.
Small chunks improve precision but may lose context.

## 7.3 Cost vs Performance

Cloud APIs may provide stronger models, but local Ollama inference reduces cost and improves privacy.

# 8 Testing Strategy

## 8.1 Testing Approach

System tested using sample customer support queries across multiple scenarios.

## 8.2 Sample Queries

- How do I track my order?
- What is your refund policy?
- How can I cancel my shipment?
- How long does delivery take?
- Can I return a damaged item?

## 8.3 Validation Metrics

- Relevance of retrieved chunks
- Accuracy of final response
- Confidence score behavior
- HITL trigger correctness

# 9 Future Enhancements

- Multi-document support
- Feedback learning loop
- Memory integration
- Streamlit Web UI
- REST API deployment
- Admin dashboard
- Voice support assistant
- Query analytics

# 10 Summary

The RAG Customer Support Assistant combines semantic retrieval, local language models, workflow routing, and human supervision to automate customer support in a reliable and scalable manner. It improves speed, reduces manual effort, and ensures trustworthy responses.

*Done by*

**Bhargav S**